

HW2-5 作业感受（第二次挑战）

（1）如何完成第二次挑战

主体流程和第一次类似：先把 SGLang 0.5.14 的 CPU 服务跑起来，再按任务书把测量脚本写好。AI 代理负责搭环境、写脚本初稿和读源码定位函数，我负责对照任务书核对硬性参数（每组 32 条、并发 8、`sampling_seed=2026`、`ignore_eos`、`max_new_tokens=16` 这些一个都不能错），每组跑完就打开 `per_request.jsonl` 抽查几条，看 `input/output tokens`、`cached_tokens` 对不对得上，不对就回去查脚本而不是先怀疑机器。迭代下来最大的体会是验收不能偷懒，有一轮脚本残留了 32 条 `timeout` 记录在 `jsonl` 里，就是逐行翻文件才发现的。

（2）最困难的部分

最折腾的是流式解析的 bug。第一版脚本把响应体当纯 JSON 行去解析，结果 32 条请求全部 `timeout`，一开始还以为是 CPU 推理太慢扛不住并发。后来用 `curl` 直接抓原始响应，看到每行前面带着 `data:` 前缀，才知道这是 SSE 格式，解析层先剥掉前缀就通了。这件事给我的教训是：调网络接口的问题，第一步永远是先看原始字节，别急着改业务逻辑。另一个困难是 CPU 上跑 512 token 的 `prefill` 要二十多秒，一组实验两三分钟，试错成本很高，逼着我在每次重跑前把参数全部检查一遍再发请求。

（3）对科研的启发

第一，先跑通最小闭环再谈优化，一个 32 条请求的小实验跑通了，比一个设计了很久但没跑起来的框架有价值。第二，数据要能溯源：报告里每个数字都应该能对到原始 `jsonl` 里的某一行，这次对照表就是从逐请求记录重新算出来的，心里很踏实。第三，不能全信 AI：这次它把 `get_new_batch_prefill` 的位置标到了 `schedule_policy.py`，我翻 v0.5.14 源码才发现函数其实在 `scheduler.py` 里，AI 给的结论必须自己验证过才能写进报告。第四，资源受限时找替代路径也是一种能力，没有 GPU 就用 CPU 加 `torch_native`，速度慢但对照组的结论方向照样成立。